Eksplorasi, Perbandingan, dan Analisis 6 Arsitektur CNN pada CIFAR-10

Mata Kuliah: Deep Learning (IF25-40401) — Institut Teknologi Sumatera **Pertemuan 5:** CNN dan Variasinya

Nama	Tengku Hafid Diraputra
NIM	123140043
Nama	Ahmat Prayoga Sembiring
NIM	123140053
Kelas	DeepLearning RB

Spesifikasi eksperimen (sesuai ketentuan tugas)

Ketentuan	Nilai
Dataset	CIFAR-10, pembagian **default torchvision**: `train=True` (50.000 citra) dan `train=False` (10.000 citra)
Random seed	**42**
Batch size	**64**
Optimizer	**Adam**, learning rate **0,001**
Epoch	**10**
Metrik evaluasi	**Akurasi** pada data uji
Model	LeNet-5, AlexNet, VGG-16, GoogLeNet, **ResNet-18**, **MobileNetV2** (2 model pilihan)

1. Konfigurasi dan Setup

```
CONFIG = {
    "SEED": 42,
    "BATCH_SIZE": 64,
    "LR": 0.001,
    "EPOCHS": 10,
    "AMP": True,            # mixed precision di GPU: ±2x lebih cepat, hyperparameter tidak berubah
    "DIR_DATA": "./data",
    "DIR_HASIL": "hasil",   # tabel, grafik, dan riwayat training disimpan di sini
}
MODEL_DIPAKAI = ["LeNet-5", "AlexNet", "VGG-16", "GoogLeNet", "ResNet-18", "MobileNetV2"]

# Opsional: simpan hasil ke Google Drive agar tidak hilang bila
```

```
runtime Colab ter-reset.
# from google.colab import drive; drive.mount("/content/drive")
# CONFIG["DIR_HASIL"] = "/content/drive/MyDrive/tugas_cnn_cifar10"

import os, json, time, random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
from torchvision import datasets
from torch.utils.flop_counter import FlopCounterMode

def set_seed(seed):
    """Mengunci semua sumber keacakan: Python, NumPy, PyTorch (CPU &
GPU), dan cuDNN."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

set_seed(CONFIG["SEED"])
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
USE_AMP = CONFIG["AMP"] and device.type == "cuda"
os.makedirs(CONFIG["DIR_HASIL"], exist_ok=True)

# Palet warna tetap per model (urutan tidak berubah di semua grafik) +
bentuk penanda berbeda
WARNA = dict(zip(MODEL_DIPAKAI, ["#2a78d6", "#eb6834", "#1baf7a",
"#eda100", "#e87ba4", "#008300"]))
PENANDA = dict(zip(MODEL_DIPAKAI, ["o", "s", "^", "D", "v", "P"]))
plt.rcParams.update({"figure.dpi": 110, "axes.spines.top": False,
"axes.spines.right": False,
                     "axes.grid": True, "grid.alpha": 0.3,
"font.size": 10})

print("=" * 62)
print(f"PyTorch / torchvision : {torch.__version__} /
{torchvision.__version__}")
print(f"Perangkat             : {device}" + (f"
({torch.cuda.get_device_name(0)})" if device.type == "cuda" else ""))
print(f"Mixed precision (AMP) : {USE_AMP}")
print("Konfigurasi           :", {k: v for k, v in CONFIG.items() if k
in ("SEED", "BATCH_SIZE", "LR", "EPOCHS")})
print("=" * 62)
if device.type != "cuda":
```

```
    print("PERINGATAN: GPU tidak aktif. Di Colab: Runtime -> Change
runtime type -> T4 GPU")

============================================================
PyTorch / torchvision : 2.11.0+cu130 / 0.26.0+cu130
Perangkat             : cuda (Tesla T4)
Mixed precision (AMP) : True
Konfigurasi           : {'SEED': 42, 'BATCH_SIZE': 64, 'LR': 0.001,
'EPOCHS': 10}
============================================================
```

2. Eksplorasi Dataset CIFAR-10

Dataset dimuat dengan **pembagian default torchvision**: `train=True` untuk data latih dan `train=False` untuk data uji. Tidak ada data yang dipindahkan antar-split.

```
train_set = datasets.CIFAR10(CONFIG["DIR_DATA"], train=True,
download=True)
test_set = datasets.CIFAR10(CONFIG["DIR_DATA"], train=False,
download=True)
KELAS = train_set.classes

print(f"Data latih : {len(train_set):,} citra | shape array
{train_set.data.shape} | dtype {train_set.data.dtype}")
print(f"Data uji   : {len(test_set):,} citra | shape array
{test_set.data.shape}")
print(f"Jumlah kelas: {len(KELAS)} -> {KELAS}")

jumlah_latih = np.bincount(train_set.targets, minlength=10)
jumlah_uji = np.bincount(test_set.targets, minlength=10)
print("\nDistribusi kelas (latih / uji):")
for k, a, b in zip(KELAS, jumlah_latih, jumlah_uji):
    print(f"  {k:<11}: {a:>5,} / {b:>5,}")
print("-> Dataset seimbang: setiap kelas punya jumlah citra yang sama,
sehingga akurasi adalah metrik yang adil.")

100%|██████████| 170M/170M [00:31<00:00, 5.36MB/s]

Data latih : 50,000 citra | shape array (50000, 32, 32, 3) | dtype
uint8
Data uji   : 10,000 citra | shape array (10000, 32, 32, 3)
Jumlah kelas: 10 -> ['airplane', 'automobile', 'bird', 'cat', 'deer',
'dog', 'frog', 'horse', 'ship', 'truck']

Distribusi kelas (latih / uji):
  airplane   : 5,000 / 1,000
  automobile : 5,000 / 1,000
  bird       : 5,000 / 1,000
  cat        : 5,000 / 1,000
```

```
  deer       : 5,000 / 1,000
  dog        : 5,000 / 1,000
  frog       : 5,000 / 1,000
  horse      : 5,000 / 1,000
  ship       : 5,000 / 1,000
  truck      : 5,000 / 1,000
-> Dataset seimbang: setiap kelas punya jumlah citra yang sama,
sehingga akurasi adalah metrik yang adil.

rng = np.random.default_rng(CONFIG["SEED"])
fig, axs = plt.subplots(2, 10, figsize=(15, 3.4))
for c in range(10):
    idx = rng.choice(np.where(np.array(train_set.targets) == c)[0],
size=2, replace=False)
    for r in range(2):
        axs[r, c].imshow(train_set.data[idx[r]])
        axs[r, c].axis("off")
    axs[0, c].set_title(KELAS[c], fontsize=9)
plt.suptitle("Dua contoh citra per kelas (resolusi asli 32×32
piksel)", y=1.02)
plt.tight_layout()
plt.savefig(os.path.join(CONFIG["DIR_HASIL"], "contoh_data.png"),
bbox_inches="tight")
plt.show()
```



Normalisasi

Mean dan standar deviasi per kanal dihitung **hanya dari data latih**, lalu dipakai untuk menormalkan data latih dan data uji. Data uji tidak boleh dipakai untuk menghitung statistik apa pun. Hasilnya setara dengan `transforms.ToTensor()` + `transforms.Normalize(mean, std)`.

```
def ke_tensor(ds):
    x = torch.from_numpy(ds.data).permute(0, 3, 1, 2).contiguous()   #
(N, 32, 32, 3) -> (N, 3, 32, 32), uint8
    return x, torch.tensor(ds.targets)

x_train, y_train = ke_tensor(train_set)
x_test, y_test = ke_tensor(test_set)
```

```
MEAN = torch.stack([x_train[:, c].float().div(255).mean() for c in
range(3)])
STD = torch.stack([x_train[:, c].float().div(255).std() for c in
range(3)])
print("Mean per kanal (R, G, B):", MEAN.numpy().round(4))
print("Std  per kanal (R, G, B):", STD.numpy().round(4))
```

```
Mean per kanal (R, G, B): [0.4914 0.4822 0.4465]
Std  per kanal (R, G, B): [0.247  0.2435 0.2616]
```

DataLoader

Seluruh dataset (±180 MB dalam format `uint8`) disimpan di GPU, lalu dinormalisasi per batch. Cara ini jauh lebih cepat daripada `DataLoader` biasa di Colab yang CPU-nya hanya 2 core, dan hasilnya identik. Urutan acak (shuffle) dikendalikan oleh `torch.Generator` dengan seed 42 yang dibuat ulang untuk setiap model, sehingga **keenam model melihat urutan batch yang persis sama**.

```
class CIFARLoader:
    def __init__(self, x, y, batch_size, shuffle, seed=None):
        self.x, self.y = x.to(device), y.to(device)
        self.bs, self.shuffle = batch_size, shuffle
        self.gen = torch.Generator().manual_seed(seed) if shuffle else
None
        self.mean = MEAN.view(1, 3, 1, 1).to(device)
        self.std = STD.view(1, 3, 1, 1).to(device)

    def __len__(self):
        return (len(self.x) + self.bs - 1) // self.bs

    def __iter__(self):
        n = len(self.x)
        idx = torch.randperm(n, generator=self.gen) if self.shuffle
else torch.arange(n)
        idx = idx.to(device)
        for k in range(0, n, self.bs):
            b = idx[k:k + self.bs]
            x = (self.x[b].float() / 255.0 - self.mean) / self.std
            yield x, self.y[b]

def buat_loader(seed):
    train_loader = CIFARLoader(x_train, y_train, CONFIG["BATCH_SIZE"],
shuffle=True, seed=seed)
    test_loader = CIFARLoader(x_test, y_test, 1000, shuffle=False)
    return train_loader, test_loader

_tl, _ = buat_loader(CONFIG["SEED"])
xb, yb = next(iter(_tl))
print(f"Satu batch: x {tuple(xb.shape)} {xb.dtype}, y
```

```
{tuple(yb.shape)} | "
      f"mean {xb.mean():.3f}, std {xb.std():.3f} | {len(_tl)} iterasi
per epoch")
del _tl, xb, yb
```

```
Satu batch: x (64, 3, 32, 32) torch.float32, y (64,) | mean -0.057,
std 0.927 | 782 iterasi per epoch
```

3. Alat Ukur Model

- **Parameter**: jumlah bobot yang dilatih (`p.numel()`).
- **MAC (multiply-accumulate)**: jumlah operasi kali-tambah untuk satu citra. `FlopCounterMode` menghitung FLOP = 2 × MAC, jadi hasilnya dibagi 2 (sama seperti notebook praktikum 04).
- **Porsi parameter di FC**: menguji pola "parameter menumpuk di fully connected" (slide 8 dan 14).

```
def hitung_param(model):
    return sum(p.numel() for p in model.parameters())

def hitung_param_fc(model):
    return sum(p.numel() for m in model.modules() if isinstance(m,
nn.Linear) for p in m.parameters())

@torch.no_grad()
def hitung_mac(model, input_size=(1, 3, 32, 32)):
    model.eval()
    x = torch.randn(*input_size,
device=next(model.parameters()).device)
    with FlopCounterMode(display=False) as fc:
        model(x)
    return fc.get_total_flops() / 2

LEWATI = (nn.ReLU, nn.ReLU6, nn.Tanh, nn.BatchNorm2d, nn.Dropout,
nn.Flatten, nn.Identity)

@torch.no_grad()
def jejak_shape(model, input_size=(1, 3, 32, 32)):
    """Mencatat shape output tiap layer/blok utama (aktivasi, BN,
dropout dilewati)."""
    target = []
    for nama, anak in model.named_children():
        if isinstance(anak, nn.Sequential) and len(anak) > 1:
            target += [(f"{nama}.{n}", m) for n, m in
anak.named_children() if not isinstance(m, LEWATI)]
        elif not isinstance(anak, LEWATI):
            target.append((nama, anak))
    log, hooks = [], []
    for nama, m in target:
        hooks.append(m.register_forward_hook(lambda mod, i, o,
```

```
nama=nama: log.append((nama, type(mod).__name__,
tuple(o.shape[1:]))))
    model.eval()
    model(torch.randn(*input_size,
device=next(model.parameters()).device))
    for h in hooks:
        h.remove()
    return log
```

4. Perancangan dan Implementasi Arsitektur

Semua arsitektur ditulis sendiri dengan `nn.Module`. Ringkasan adaptasi untuk citra 32×32:

Model	Tahun	Ide kunci (slide)	Yang dipertahankan	Yang diadaptasi untuk 32×32
LeNet-5	1998	conv → pool → FC (P4)	Seluruh arsitektur: Tanh + AvgPool, 6/16 filter 5×5, FC 120-84	C1 menerima 3 kanal (RGB)
AlexNet	2012	ReLU, Dropout, max-pool overlap (sl. 7–11)	5 conv + 3 FC, kanal 64-192-384-256-256, FC 4096-4096, Dropout 0,5	conv1 11×11 stride 4 → 5×5 stride 1
VGG-16	2014	Tumpukan conv 3×3 seragam (sl. 12–15)	13 conv 3×3 dalam 5 blok, FC 4096-4096	FC6 menerima 512 (bukan 7×7×512); **BatchNorm** ditambahkan (varian `vgg16_bn`, sl. 15)
GoogLeNet	2014	Inception, bottleneck 1×1, GAP (sl. 16–17)	9 modul Inception dengan kanal paper (termasuk cabang 5×5 asli), GAP	Stem 7×7/s2 + 2 max-pool → 1 conv 3×3; BatchNorm; tanpa auxiliary classifier
ResNet-18	2015	Residual connection (sl. 20–25)	4 stage × 2 basic block, kanal 64-512, projection shortcut, GAP	Stem 7×7/s2 + max-pool → conv 3×3/s1
MobileNetV2	2018	Depthwise separable, inverted residual	Konfigurasi blok (t, c, n, s) paper, ReLU6, linear	Stride stem dan stage kanal-24: 2 → 1

Model	Tahun	Ide kunci (slide)	Yang dipertahankan	Yang diadaptasi untuk 32×32
		(sl. 28–31)	bottleneck	

Mengapa VGG-16 memakai BatchNorm? dikarnakan VGG tanpa BN harus dilatih bertahap (11 → 16 layer). Dilatih dari nol sekaligus dengan Adam lr 0,001, VGG-16 polos berisiko macet di akurasi ±10%.sehingga kami membuat Varian dengan BN (`vgg16_bn`) dan juga tetap memakai tumpukan 3×3 yang sama, jadi ciri khas VGG tidak hilang.

4.1 LeNet-5 (1998)

```
class LeNet5(nn.Module):
    """LeNet-5 (LeCun, 1998) — versi klasik notebook 02: Tanh + AvgPool.
    Satu-satunya perubahan: C1 menerima 3 kanal (RGB), bukan 1 kanal."""
    def __init__(self, num_classes=10):
        super().__init__()
        self.feature_extractor = nn.Sequential(
            nn.Conv2d(3, 6, kernel_size=5), nn.Tanh(),       # C1: 32 -> 28
            nn.AvgPool2d(2, 2),                               # S2: 28 -> 14
            nn.Conv2d(6, 16, kernel_size=5), nn.Tanh(),      # C3: 14 -> 10
            nn.AvgPool2d(2, 2),                               # S4: 10 -> 5
        )
        self.classifier = nn.Sequential(
            nn.Linear(16 * 5 * 5, 120), nn.Tanh(),            # C5
            nn.Linear(120, 84), nn.Tanh(),                    # F6
            nn.Linear(84, num_classes),                       # output (logits)
        )

    def forward(self, x):
        return self.classifier(torch.flatten(self.feature_extractor(x), 1))
```

4.2 AlexNet (2012)

```
class AlexNet(nn.Module):
    """AlexNet (2012) versi CIFAR. Dipertahankan: 5 conv + 3 FC, ReLU, max-pool 3x3 stride 2
    (overlapping), Dropout 0.5, FC 4096-4096, kanal 64-192-384-256-256 (versi torchvision).
    Diadaptasi: conv1 11x11/s4 -> 5x5/s1 dan conv2 tetap 5x5, karena input hanya 32x32."""
    def __init__(self, num_classes=10):
        super().__init__()
```

```
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=5, stride=1, padding=2),
nn.ReLU(inplace=True),   # 32
            nn.MaxPool2d(3, 2),
# 32 -> 15
            nn.Conv2d(64, 192, kernel_size=5, padding=2),
nn.ReLU(inplace=True),          # 15
            nn.MaxPool2d(3, 2),
# 15 -> 7
            nn.Conv2d(192, 384, kernel_size=3, padding=1),
nn.ReLU(inplace=True),
            nn.Conv2d(384, 256, kernel_size=3, padding=1),
nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=3, padding=1),
nn.ReLU(inplace=True),         # 7
            nn.MaxPool2d(3, 2),
# 7 -> 3
        )
        self.classifier = nn.Sequential(
            nn.Dropout(0.5), nn.Linear(256 * 3 * 3, 4096),
nn.ReLU(inplace=True),      # FC6
            nn.Dropout(0.5), nn.Linear(4096, 4096),
nn.ReLU(inplace=True),             # FC7
            nn.Linear(4096, num_classes),
# FC8
        )

    def forward(self, x):
        return self.classifier(torch.flatten(self.features(x), 1))
```

4.3 VGG-16 (2014)

```
VGG16_CFG = [(2, 64), (2, 128), (3, 256), (3, 512), (3, 512)]   #
(jumlah conv, kanal) per blok

def vgg_block(c_in, c_out, n_conv):
    """Blok VGG (slide 15) + BatchNorm setelah tiap conv (varian
vgg16_bn)."""
    layers = []
    for i in range(n_conv):
        c = c_in if i == 0 else c_out
        layers += [nn.Conv2d(c, c_out, 3, padding=1),
nn.BatchNorm2d(c_out), nn.ReLU(inplace=True)]
    layers.append(nn.MaxPool2d(2, 2))                    # resolusi /
2
    return nn.Sequential(*layers)

class VGG16(nn.Module):
    """VGG-16-BN (2014) versi CIFAR. 13 conv 3x3 identik dengan paper;
setelah 5 blok,
```

```
    32x32 menjadi 1x1x512 sehingga FC6 menerima 512 (bukan
7x7x512)."""
    def __init__(self, num_classes=10):
        super().__init__()
        blok, c_in = [], 3
        for n_conv, c_out in VGG16_CFG:
            blok.append(vgg_block(c_in, c_out, n_conv))
            c_in = c_out
        self.features = nn.Sequential(*blok)
        self.classifier = nn.Sequential(
            nn.Linear(512, 4096), nn.ReLU(inplace=True),
nn.Dropout(0.5),    # FC6
            nn.Linear(4096, 4096), nn.ReLU(inplace=True),
nn.Dropout(0.5),   # FC7
            nn.Linear(4096, num_classes),
# FC8
        )

    def forward(self, x):
        return self.classifier(torch.flatten(self.features(x), 1))
```

4.4 GoogLeNet (2014)

```
class BasicConv2d(nn.Module):
    """Conv (tanpa bias) -> BatchNorm -> ReLU (pola baku slide 19)."""
    def __init__(self, c_in, c_out, **kwargs):
        super().__init__()
        self.conv = nn.Conv2d(c_in, c_out, bias=False, **kwargs)
        self.bn = nn.BatchNorm2d(c_out, eps=0.001)

    def forward(self, x):
        return F.relu(self.bn(self.conv(x)), inplace=True)

class Inception(nn.Module):
    """Modul Inception (slide 17): 4 cabang paralel, digabung di
dimensi kanal."""
    def __init__(self, c_in, c1, c3_red, c3, c5_red, c5, c_pool):
        super().__init__()
        self.branch1 = BasicConv2d(c_in, c1, kernel_size=1)
        self.branch2 = nn.Sequential(BasicConv2d(c_in, c3_red,
kernel_size=1),          # 1x1 bottleneck
                                     BasicConv2d(c3_red, c3,
kernel_size=3, padding=1))
        self.branch3 = nn.Sequential(BasicConv2d(c_in, c5_red,
kernel_size=1),          # 1x1 bottleneck
                                     BasicConv2d(c5_red, c5,
kernel_size=5, padding=2))  # 5x5 sesuai paper
        self.branch4 = nn.Sequential(nn.MaxPool2d(3, stride=1,
padding=1),
                                     BasicConv2d(c_in, c_pool,
```

```
kernel_size=1))

    def forward(self, x):
        return torch.cat([self.branch1(x), self.branch2(x),
self.branch3(x), self.branch4(x)], dim=1)

class GoogLeNet(nn.Module):
    """GoogLeNet (2014) versi CIFAR. 9 modul Inception dengan
konfigurasi kanal paper + GAP.
    Diadaptasi: stem 7x7/s2 + 2 max-pool diganti satu conv 3x3
(resolusi tetap 32x32),
    BatchNorm ditambahkan, auxiliary classifier tidak dipakai."""
    def __init__(self, num_classes=10):
        super().__init__()
        self.stem = BasicConv2d(3, 192, kernel_size=3, padding=1)
# 32x32x192
        self.inception3a = Inception(192, 64, 96, 128, 16, 32, 32)
# -> 256
        self.inception3b = Inception(256, 128, 128, 192, 32, 96, 64)
# -> 480
        self.maxpool3 = nn.MaxPool2d(3, stride=2, padding=1)
# 32 -> 16
        self.inception4a = Inception(480, 192, 96, 208, 16, 48, 64)
# -> 512
        self.inception4b = Inception(512, 160, 112, 224, 24, 64, 64)
        self.inception4c = Inception(512, 128, 128, 256, 24, 64, 64)
        self.inception4d = Inception(512, 112, 144, 288, 32, 64, 64)
# -> 528
        self.inception4e = Inception(528, 256, 160, 320, 32, 128, 128)
# -> 832
        self.maxpool4 = nn.MaxPool2d(3, stride=2, padding=1)
# 16 -> 8
        self.inception5a = Inception(832, 256, 160, 320, 32, 128, 128)
        self.inception5b = Inception(832, 384, 192, 384, 48, 128, 128)
# -> 1024
        self.avgpool = nn.AdaptiveAvgPool2d(1)
# Global Average Pooling
        self.dropout = nn.Dropout(0.4)
        self.fc = nn.Linear(1024, num_classes)

    def forward(self, x):
        x = self.stem(x)
        x = self.maxpool3(self.inception3b(self.inception3a(x)))
        x = self.inception4c(self.inception4b(self.inception4a(x)))
        x = self.maxpool4(self.inception4e(self.inception4d(x)))
        x = self.inception5b(self.inception5a(x))
        return self.fc(self.dropout(torch.flatten(self.avgpool(x),
1)))
```

4.5 ResNet-18 (2015) — model pilihan 1

```
class BasicBlock(nn.Module):
    """Basic block ResNet (slide 23): y = ReLU(F(x) + shortcut(x))."""
    def __init__(self, c_in, c_out, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(c_in, c_out, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(c_out)
        self.conv2 = nn.Conv2d(c_out, c_out, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(c_out)
        self.shortcut = nn.Identity()
        if stride != 1 or c_in != c_out:                            #
projection shortcut
            self.shortcut = nn.Sequential(nn.Conv2d(c_in, c_out, 1,
stride, bias=False),
                                          nn.BatchNorm2d(c_out))

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        return F.relu(out + self.shortcut(x))                       #
<-- skip connection

class ResNet18(nn.Module):
    """ResNet-18 (2015) versi CIFAR. 4 stage x 2 basic block, kanal
64-128-256-512, GAP + FC.
    Diadaptasi: stem 7x7/s2 + max-pool diganti conv 3x3/s1 (seperti
eksperimen CIFAR di paper ResNet)."""
    def __init__(self, num_classes=10):
        super().__init__()
        self.stem = nn.Sequential(nn.Conv2d(3, 64, 3, 1, 1,
bias=False), nn.BatchNorm2d(64), nn.ReLU(inplace=True))
        self.layer1 = nn.Sequential(BasicBlock(64, 64), BasicBlock(64,
64))            # 32x32
        self.layer2 = nn.Sequential(BasicBlock(64, 128, 2),
BasicBlock(128, 128))      # 16x16
        self.layer3 = nn.Sequential(BasicBlock(128, 256, 2),
BasicBlock(256, 256))     # 8x8
        self.layer4 = nn.Sequential(BasicBlock(256, 512, 2),
BasicBlock(512, 512))     # 4x4
        self.avgpool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Linear(512, num_classes)

    def forward(self, x):
        x =
self.layer4(self.layer3(self.layer2(self.layer1(self.stem(x)))))
        return self.fc(torch.flatten(self.avgpool(x), 1))
```

4.6 MobileNetV2 (2018) — model pilihan 2

```
class InvertedResidual(nn.Module):
    """Blok MobileNetV2 (slide 31): 1x1 ekspansi -> 3x3 depthwise ->
1x1 proyeksi LINEAR.
    Skip connection hanya bila stride 1 dan kanal masuk = kanal
keluar."""
    def __init__(self, c_in, c_out, stride, t):
        super().__init__()
        hidden = c_in * t
        self.use_skip = stride == 1 and c_in == c_out
        layers = []
        if t != 1:
            layers += [nn.Conv2d(c_in, hidden, 1, bias=False),
nn.BatchNorm2d(hidden), nn.ReLU6(inplace=True)]
        layers += [
            nn.Conv2d(hidden, hidden, 3, stride, 1, groups=hidden,
bias=False),   # depthwise: groups = kanal
            nn.BatchNorm2d(hidden), nn.ReLU6(inplace=True),
            nn.Conv2d(hidden, c_out, 1, bias=False),
nn.BatchNorm2d(c_out),        # pointwise, tanpa aktivasi
        ]
        self.block = nn.Sequential(*layers)

    def forward(self, x):
        return x + self.block(x) if self.use_skip else self.block(x)

class MobileNetV2(nn.Module):
    """MobileNetV2 (2018) versi CIFAR. Konfigurasi blok (t, c, n, s)
identik dengan paper.
    Diadaptasi: stride stem 2 -> 1 dan stride stage kanal-24 2 -> 1,
sehingga peta fitur akhir 4x4."""
    CFG = [(1, 16, 1, 1), (6, 24, 2, 1), (6, 32, 3, 2), (6, 64, 4, 2),
           (6, 96, 3, 1), (6, 160, 3, 2), (6, 320, 1, 1)]

    def __init__(self, num_classes=10):
        super().__init__()
        layers = [nn.Sequential(nn.Conv2d(3, 32, 3, 1, 1, bias=False),
nn.BatchNorm2d(32), nn.ReLU6(inplace=True))]
        c_in = 32
        for t, c, n, s in self.CFG:
            for i in range(n):
                layers.append(InvertedResidual(c_in, c, s if i == 0
else 1, t))
                c_in = c
        layers.append(nn.Sequential(nn.Conv2d(c_in, 1280, 1,
bias=False), nn.BatchNorm2d(1280), nn.ReLU6(inplace=True)))
        self.features = nn.Sequential(*layers)
        self.avgpool = nn.AdaptiveAvgPool2d(1)
        self.classifier = nn.Sequential(nn.Dropout(0.2),
```

```
nn.Linear(1280, num_classes))

    def forward(self, x):
        return
self.classifier(torch.flatten(self.avgpool(self.features(x)), 1))

PEMBUAT_MODEL = {
    "LeNet-5": LeNet5, "AlexNet": AlexNet, "VGG-16": VGG16,
    "GoogLeNet": GoogLeNet, "ResNet-18": ResNet18, "MobileNetV2":
MobileNetV2,
}
INFO_MODEL = {
    "LeNet-5":     (1998, "conv-pool-FC klasik, Tanh"),
    "AlexNet":     (2012, "ReLU, Dropout, model besar"),
    "VGG-16":      (2014, "tumpukan conv 3x3 seragam"),
    "GoogLeNet":   (2014, "Inception, 1x1, GAP"),
    "ResNet-18":   (2015, "residual connection"),
    "MobileNetV2": (2018, "depthwise separable, inverted residual"),
}
```

4.7 Verifikasi arsitektur: shape, parameter, dan komputasi

Sebelum training, setiap model diuji dengan input dummy 32×32×3. Output harus `(1, 10)`. LeNet-5 harus tepat **62.006** parameter (61.706 di slide + 300, karena C1 menerima 3 kanal).

```
assert hitung_param(LeNet5()) == 62_006, "LeNet-5 tidak sesuai
hitungan analitis"

ringkasan_arsitektur = {}
for nama in MODEL_DIPAKAI:
    m = PEMBUAT_MODEL[nama]()
    p, pfc = hitung_param(m), hitung_param_fc(m)
    mac = hitung_mac(m)
    out = m(torch.randn(1, 3, 32, 32)).shape
    assert tuple(out) == (1, 10)
    ringkasan_arsitektur[nama] = dict(param=p, param_fc=pfc, mac=mac)
    jejak = jejak_shape(m)
    print(f"\n=== {nama} | {p:,} parameter | {mac/1e6:,.1f} juta MAC |
{pfc/p:.1%} parameter di FC ===")
    for i in range(0, len(jejak), 3):
        print("   " + " | ".join(f"{n} -> {'x'.join(map(str, s))}" for
n, _, s in jejak[i:i + 3]))
    del m


=== LeNet-5 | 62,006 parameter | 0.7 juta MAC | 95.4% parameter di FC
===
   feature_extractor.0 -> 6x28x28 | feature_extractor.2 -> 6x14x14 |
feature_extractor.3 -> 16x10x10
   feature_extractor.5 -> 16x5x5 | classifier.0 -> 120 | classifier.2
```

```
-> 84
   classifier.4 -> 10

=== AlexNet | 28,714,826 parameter | 205.1 juta MAC | 91.5% parameter
di FC ===
   features.0 -> 64x32x32 | features.2 -> 64x15x15 | features.3 ->
192x15x15
   features.5 -> 192x7x7 | features.6 -> 384x7x7 | features.8 ->
256x7x7
   features.10 -> 256x7x7 | features.12 -> 256x3x3 | classifier.1 ->
4096
   classifier.4 -> 4096 | classifier.6 -> 10

=== VGG-16 | 33,646,666 parameter | 332.1 juta MAC | 56.2% parameter
di FC ===
   features.0 -> 64x16x16 | features.1 -> 128x8x8 | features.2 ->
256x4x4
   features.3 -> 512x2x2 | features.4 -> 512x1x1 | classifier.0 ->
4096
   classifier.3 -> 4096 | classifier.6 -> 10

=== GoogLeNet | 5,871,914 parameter | 1,447.2 juta MAC | 0.2%
parameter di FC ===
   stem -> 192x32x32 | inception3a -> 256x32x32 | inception3b ->
480x32x32
   maxpool3 -> 480x16x16 | inception4a -> 512x16x16 | inception4b ->
512x16x16
   inception4c -> 512x16x16 | inception4d -> 528x16x16 | inception4e -
> 832x16x16
   maxpool4 -> 832x8x8 | inception5a -> 832x8x8 | inception5b ->
1024x8x8
   avgpool -> 1024x1x1 | fc -> 10

=== ResNet-18 | 11,173,962 parameter | 555.4 juta MAC | 0.0% parameter
di FC ===
   stem.0 -> 64x32x32 | layer1.0 -> 64x32x32 | layer1.1 -> 64x32x32
   layer2.0 -> 128x16x16 | layer2.1 -> 128x16x16 | layer3.0 -> 256x8x8
   layer3.1 -> 256x8x8 | layer4.0 -> 512x4x4 | layer4.1 -> 512x4x4
   avgpool -> 512x1x1 | fc -> 10

=== MobileNetV2 | 2,236,682 parameter | 88.0 juta MAC | 0.6% parameter
di FC ===
   features.0 -> 32x32x32 | features.1 -> 16x32x32 | features.2 ->
24x32x32
   features.3 -> 24x32x32 | features.4 -> 32x16x16 | features.5 ->
32x16x16
   features.6 -> 32x16x16 | features.7 -> 64x8x8 | features.8 ->
64x8x8
   features.9 -> 64x8x8 | features.10 -> 64x8x8 | features.11 ->
96x8x8
```

```
   features.12 -> 96x8x8 | features.13 -> 96x8x8 | features.14 ->
160x4x4
   features.15 -> 160x4x4 | features.16 -> 160x4x4 | features.17 ->
320x4x4
   features.18 -> 1280x4x4 | avgpool -> 1280x1x1 | classifier.1 -> 10
```

5. Pelatihan

Fungsi pelatihan memakai **5 langkah baku** (slide P4 / notebook 02): `zero_grad` → forward → loss → `backward` → `step`. Setiap model dimulai dari kondisi yang sama: `set_seed(42)` sebelum model dibuat, generator shuffle dengan seed 42, optimizer `Adam(lr=0.001)`, `CrossEntropyLoss`, batch 64, 10 epoch.

Karena pembagian default torchvision hanya terdiri dari *train* dan *test*, akurasi uji dicatat di setiap epoch untuk **memantau kurva belajar**. Metrik resmi perbandingan adalah **akurasi uji pada epoch ke-10**, bukan akurasi tertinggi. Memilih epoch terbaik berdasarkan data uji sama saja dengan "mengintip" data uji.

```
def autocast():
    return torch.autocast(device_type=device.type,
dtype=torch.float16, enabled=USE_AMP)

@torch.no_grad()
def evaluasi(model, loader, criterion):
    model.eval()
    total_loss, benar, total, semua_pred = 0.0, 0, 0, []
    for x, y in loader:
        with autocast():
            out = model(x)
        total_loss += criterion(out.float(), y).item() * y.size(0)
        pred = out.argmax(1)
        benar += (pred == y).sum().item()
        total += y.size(0)
        semua_pred.append(pred.cpu())
    return total_loss / total, benar / total, torch.cat(semua_pred)

def latih_model(nama):
    set_seed(CONFIG["SEED"])
    model = PEMBUAT_MODEL[nama]().to(device)
    train_loader, test_loader = buat_loader(CONFIG["SEED"])
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=CONFIG["LR"])
    scaler = torch.amp.GradScaler(device.type, enabled=USE_AMP)

    riwayat = []
    print(f"\n{'=' * 78}\n{nama}: {hitung_param(model):,} parameter\
n{'=' * 78}")
    for epoch in range(1, CONFIG["EPOCHS"] + 1):
        model.train()
```

```
        t0 = time.time()
        total_loss, benar, total = 0.0, 0, 0
        for x, y in train_loader:
            optimizer.zero_grad(set_to_none=True)      # 1. reset
gradien
            with autocast():
                out = model(x)                         # 2. forward
                loss = criterion(out, y)               # 3. hitung
loss
            scaler.scale(loss).backward()              # 4. backward
            scaler.step(optimizer)                     # 5. update
bobot
            scaler.update()
            total_loss += loss.item() * y.size(0)
            benar += (out.argmax(1) == y).sum().item()
            total += y.size(0)
        if device.type == "cuda":
            torch.cuda.synchronize()
        detik = time.time() - t0
        test_loss, test_acc, pred = evaluasi(model, test_loader,
criterion)
        riwayat.append(dict(epoch=epoch, train_loss=total_loss /
total, train_acc=benar / total,
                            test_loss=test_loss, test_acc=test_acc,
detik=detik))
        print(f"Epoch {epoch:2d}/{CONFIG['EPOCHS']} | train loss
{total_loss/total:.4f} acc {benar/total:6.2%} "
              f"| test loss {test_loss:.4f} acc {test_acc:6.2%} |
{detik:5.1f}s")

    hasil = dict(nama=nama, riwayat=riwayat, pred_test=pred.tolist(),
**ringkasan_arsitektur[nama])
    with open(os.path.join(CONFIG["DIR_HASIL"], f"{nama}.json"), "w")
as f:
        json.dump(hasil, f)
    del model, optimizer
    if device.type == "cuda":
        torch.cuda.empty_cache()
    return hasil
```

Jalankan pelatihan keenam model. Bila runtime terputus di tengah jalan, jalankan ulang sel ini. Model yang hasilnya sudah tersimpan di folder `hasil/` akan dimuat, tidak dilatih ulang.

```
HASIL = {}
t_mulai = time.time()
for nama in MODEL_DIPAKAI:
    path = os.path.join(CONFIG["DIR_HASIL"], f"{nama}.json")
    if os.path.exists(path):
        with open(path) as f:
```

```
            HASIL[nama] = json.load(f)
        print(f"{nama}: hasil tersimpan dimuat dari {path}")
    else:
        HASIL[nama] = latih_model(nama)
print(f"\nTotal waktu: {(time.time() - t_mulai) / 60:.1f} menit")


================================================================================
LeNet-5: 62,006 parameter
================================================================================
Epoch  1/10 | train loss 1.7211 acc 38.87% | test loss 1.5481 acc 45.00% |   6.0s
Epoch  2/10 | train loss 1.4503 acc 48.03% | test loss 1.3938 acc 50.15% |   2.9s
Epoch  3/10 | train loss 1.3190 acc 52.74% | test loss 1.3018 acc 53.23% |   2.4s
Epoch  4/10 | train loss 1.2303 acc 55.91% | test loss 1.2684 acc 54.69% |   2.4s
Epoch  5/10 | train loss 1.1693 acc 58.08% | test loss 1.2529 acc 54.93% |   2.4s
Epoch  6/10 | train loss 1.1188 acc 60.22% | test loss 1.2187 acc 56.58% |   2.4s
Epoch  7/10 | train loss 1.0765 acc 61.71% | test loss 1.2394 acc 55.77% |   2.9s
Epoch  8/10 | train loss 1.0385 acc 63.08% | test loss 1.2132 acc 56.83% |   2.4s
Epoch  9/10 | train loss 1.0040 acc 64.19% | test loss 1.2038 acc 57.41% |   2.4s
Epoch 10/10 | train loss 0.9727 acc 65.40% | test loss 1.2153 acc 57.43% |   2.4s

================================================================================
AlexNet: 28,714,826 parameter
================================================================================
Epoch  1/10 | train loss 1.7562 acc 32.79% | test loss 1.5461 acc 43.19% |  15.3s
Epoch  2/10 | train loss 1.4297 acc 47.66% | test loss 1.3202 acc 52.79% |  15.5s
Epoch  3/10 | train loss 1.2775 acc 54.47% | test loss 1.2381 acc 56.07% |  15.0s
Epoch  4/10 | train loss 1.1850 acc 58.23% | test loss 1.1178 acc 60.94% |  15.1s
Epoch  5/10 | train loss 1.1067 acc 61.34% | test loss 1.0700 acc 63.19% |  15.1s
Epoch  6/10 | train loss 1.0526 acc 63.22% | test loss 1.0868 acc 62.58% |  15.2s
```

```
Epoch  7/10 | train loss 1.0007 acc 65.22% | test loss 0.9818 acc
65.97% |  15.2s
Epoch  8/10 | train loss 0.9617 acc 66.63% | test loss 0.9583 acc
66.74% |  15.3s
Epoch  9/10 | train loss 0.9242 acc 68.00% | test loss 0.9438 acc
67.16% |  16.5s
Epoch 10/10 | train loss 0.8921 acc 69.01% | test loss 0.9699 acc
66.60% |  15.6s

================================================================================
VGG-16: 33,646,666 parameter
================================================================================
Epoch  1/10 | train loss 1.9398 acc 21.74% | test loss 1.7710 acc
30.11% |  27.2s
Epoch  2/10 | train loss 1.6183 acc 34.99% | test loss 1.5138 acc
39.45% |  25.1s
Epoch  3/10 | train loss 1.3935 acc 47.38% | test loss 1.4182 acc
47.42% |  24.1s
Epoch  4/10 | train loss 1.1629 acc 57.83% | test loss 1.0465 acc
62.44% |  24.2s
Epoch  5/10 | train loss 0.9604 acc 66.99% | test loss 1.0694 acc
64.73% |  24.4s
Epoch  6/10 | train loss 0.8202 acc 72.44% | test loss 0.9759 acc
67.22% |  24.1s
Epoch  7/10 | train loss 0.7156 acc 76.34% | test loss 0.7443 acc
75.56% |  24.5s
Epoch  8/10 | train loss 0.6188 acc 79.80% | test loss 0.6847 acc
78.00% |  24.2s
Epoch  9/10 | train loss 0.5430 acc 82.35% | test loss 0.6702 acc
79.01% |  24.2s
Epoch 10/10 | train loss 0.4809 acc 84.69% | test loss 0.6529 acc
79.64% |  24.2s

================================================================================
GoogLeNet: 5,871,914 parameter
================================================================================
Epoch  1/10 | train loss 1.2440 acc 54.37% | test loss 0.9313 acc
67.24% |  88.7s
Epoch  2/10 | train loss 0.7655 acc 73.01% | test loss 0.7869 acc
72.44% |  87.7s
Epoch  3/10 | train loss 0.5910 acc 79.45% | test loss 0.7352 acc
75.73% |  87.8s
Epoch  4/10 | train loss 0.4787 acc 83.51% | test loss 0.5241 acc
81.83% |  87.7s
Epoch  5/10 | train loss 0.4054 acc 86.06% | test loss 0.5615 acc
81.18% |  87.8s
```

```
Epoch  6/10 | train loss 0.3452 acc 88.09% | test loss 0.5554 acc
81.90% |  87.5s
Epoch  7/10 | train loss 0.2863 acc 90.19% | test loss 0.5223 acc
83.44% |  87.3s
Epoch  8/10 | train loss 0.2359 acc 91.83% | test loss 0.4994 acc
84.40% |  87.7s
Epoch  9/10 | train loss 0.2026 acc 93.04% | test loss 0.5394 acc
83.80% |  87.2s
Epoch 10/10 | train loss 0.1644 acc 94.17% | test loss 0.6196 acc
83.98% |  87.2s

================================================================
========
ResNet-18: 11,173,962 parameter
================================================================
========
Epoch  1/10 | train loss 1.2830 acc 53.11% | test loss 1.0089 acc
64.77% |  24.5s
Epoch  2/10 | train loss 0.7958 acc 71.83% | test loss 0.8642 acc
69.69% |  24.3s
Epoch  3/10 | train loss 0.5932 acc 79.34% | test loss 0.6057 acc
78.91% |  25.2s
Epoch  4/10 | train loss 0.4605 acc 84.05% | test loss 0.5759 acc
79.88% |  24.4s
Epoch  5/10 | train loss 0.3654 acc 87.24% | test loss 0.7818 acc
74.91% |  24.4s
Epoch  6/10 | train loss 0.2793 acc 90.19% | test loss 0.5381 acc
82.98% |  24.4s
Epoch  7/10 | train loss 0.1994 acc 93.10% | test loss 0.6177 acc
81.23% |  24.4s
Epoch  8/10 | train loss 0.1448 acc 94.82% | test loss 0.5886 acc
83.43% |  24.4s
Epoch  9/10 | train loss 0.1066 acc 96.24% | test loss 0.6286 acc
83.42% |  24.4s
Epoch 10/10 | train loss 0.0860 acc 96.97% | test loss 0.6999 acc
82.64% |  24.4s

================================================================
========
MobileNetV2: 2,236,682 parameter
================================================================
========
Epoch  1/10 | train loss 1.4303 acc 47.81% | test loss 1.1553 acc
59.27% |  35.9s
Epoch  2/10 | train loss 0.9507 acc 66.41% | test loss 0.9186 acc
68.45% |  33.4s
Epoch  3/10 | train loss 0.7487 acc 73.80% | test loss 0.7132 acc
74.94% |  29.2s
Epoch  4/10 | train loss 0.6240 acc 78.49% | test loss 0.7039 acc
75.56% |  29.4s
```

```
Epoch  5/10 | train loss 0.5463 acc 80.98% | test loss 0.7223 acc
76.18% |  29.5s
Epoch  6/10 | train loss 0.4808 acc 83.09% | test loss 0.6101 acc
79.68% |  29.9s
Epoch  7/10 | train loss 0.4194 acc 85.44% | test loss 0.6355 acc
79.37% |  29.1s
Epoch  8/10 | train loss 0.3689 acc 87.10% | test loss 0.5787 acc
81.13% |  29.5s
Epoch  9/10 | train loss 0.3280 acc 88.30% | test loss 0.5966 acc
80.66% |  29.2s
Epoch 10/10 | train loss 0.2827 acc 90.06% | test loss 0.6262 acc
80.90% |  29.4s

Total waktu: 32.5 menit
```

6. Tabel Perbandingan

```
baris = []
for nama in MODEL_DIPAKAI:
    h = HASIL[nama]
    r = h["riwayat"]
    akhir = r[-1]
    terbaik = max(r, key=lambda e: e["test_acc"])
    baris.append({
        "Model": nama,
        "Tahun": INFO_MODEL[nama][0],
        "Ide kunci": INFO_MODEL[nama][1],
        "Parameter (juta)": h["param"] / 1e6,
        "% param di FC": 100 * h["param_fc"] / h["param"],
        "MAC (juta)": h["mac"] / 1e6,
        "Ukuran (MB)": h["param"] * 4 / 1e6,
        "Waktu/epoch (s)": float(np.mean([e["detik"] for e in r])),
        "Train acc (%)": 100 * akhir["train_acc"],
        "Test acc (%)": 100 * akhir["test_acc"],
        "Gap train-test": 100 * (akhir["train_acc"] -
akhir["test_acc"]),
        "Test acc maks (%)": 100 * terbaik["test_acc"],
        "Epoch maks": terbaik["epoch"],
    })
tabel = pd.DataFrame(baris)
tabel.to_csv(os.path.join(CONFIG["DIR_HASIL"],
"tabel_perbandingan.csv"), index=False)

pd.set_option("display.width", 200)
tampil = tabel.drop(columns=["Ide kunci"]).sort_values("Test acc (%)",
ascending=False)
print(f"Diurutkan berdasarkan akurasi uji epoch ke-{CONFIG['EPOCHS']}
(metrik resmi):\n")
```

```
print(tampil.to_string(index=False, float_format=lambda v:
f"{v:,.2f}"))

Diurutkan berdasarkan akurasi uji epoch ke-10 (metrik resmi):

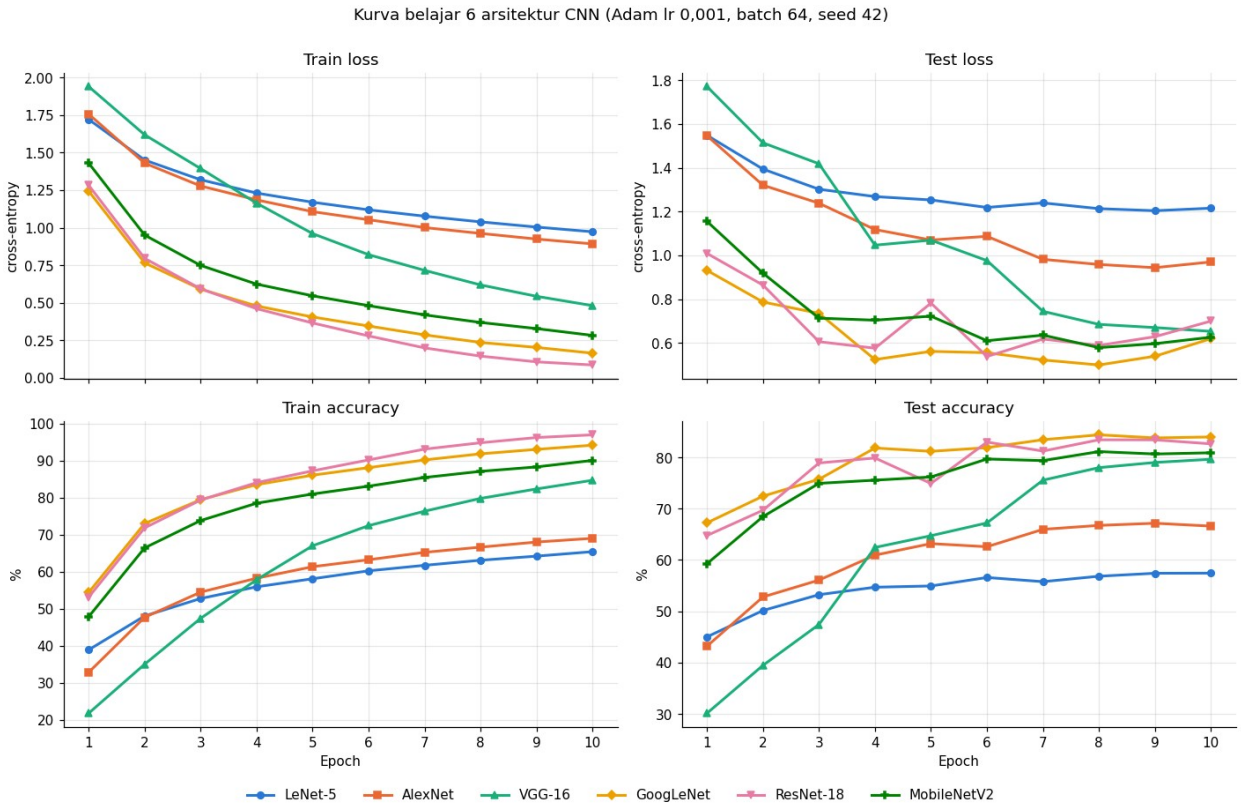
      Model  Tahun  Parameter (juta)  % param di FC  MAC (juta)
Ukuran (MB)  Waktu/epoch (s)  Train acc (%)  Test acc (%)  Gap train-
test  Test acc maks (%)  Epoch maks
  GoogLeNet   2014              5.87           0.17    1,447.24
23.49           87.65          94.17         83.98           10.19
84.40           8
  ResNet-18   2015             11.17           0.05      555.42
44.70           24.46          96.97         82.64           14.33
83.43           8
MobileNetV2   2018              2.24           0.57       87.98
8.95            30.46          90.06         80.90            9.16
81.13           8
     VGG-16   2014             33.65          56.24      332.11
134.59            24.62          84.69         79.64            5.05
79.64          10
    AlexNet   2012             28.71          91.46      205.06
114.86            15.39          69.01         66.60            2.41
67.16           9
    LeNet-5   1998              0.06          95.37        0.65
0.25              2.87          65.40         57.43            7.97
57.43          10
```

7. Visualisasi Hasil

7.1 Kurva belajar

```
ep = range(1, CONFIG["EPOCHS"] + 1)
fig, axs = plt.subplots(2, 2, figsize=(13, 8), sharex=True)
panel = [("train_loss", "Train loss", axs[0, 0]), ("test_loss", "Test
loss", axs[0, 1]),
         ("train_acc", "Train accuracy", axs[1, 0]), ("test_acc",
"Test accuracy", axs[1, 1])]
for kunci, judul, ax in panel:
    for nama in MODEL_DIPAKAI:
        nilai = [e[kunci] for e in HASIL[nama]["riwayat"]]
        if "acc" in kunci:
            nilai = [100 * v for v in nilai]
        ax.plot(ep, nilai, marker=PENANDA[nama], ms=5, lw=2,
color=WARNA[nama], label=nama)
    ax.set_title(judul)
    ax.set_ylabel("%" if "acc" in kunci else "cross-entropy")
for ax in axs[1]:
    ax.set_xlabel("Epoch")
    ax.set_xticks(list(ep))
h, l = axs[0, 0].get_legend_handles_labels()
```

```
plt.suptitle("Kurva belajar 6 arsitektur CNN (Adam lr 0,001, batch 64,
seed 42)", y=1.0)
plt.tight_layout()
fig.legend(h, l, loc="lower center", ncol=6, frameon=False,
bbox_to_anchor=(0.5, -0.04))
plt.savefig(os.path.join(CONFIG["DIR_HASIL"], "kurva_belajar.png"),
bbox_inches="tight")
plt.show()
```



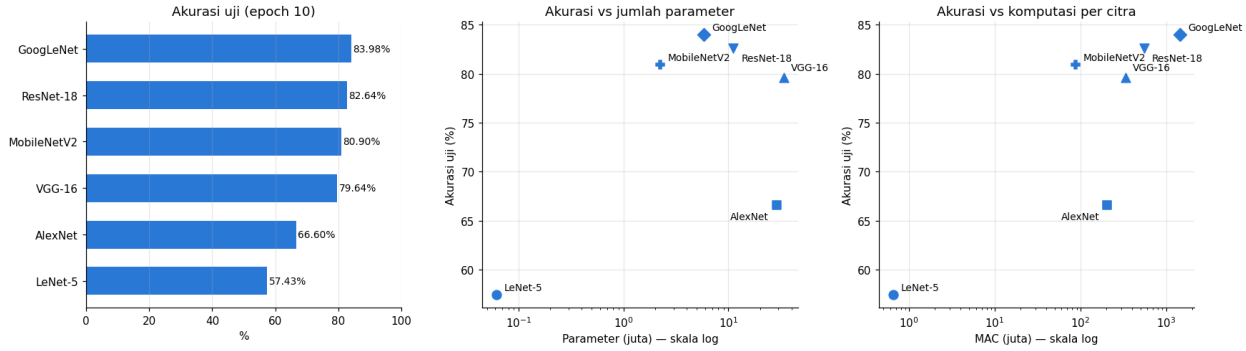
7.2 Akurasi uji dan efisiensi

```
urut = tabel.sort_values("Test acc (%)")
# posisi label bergantian agar AlexNet & VGG-16 (parameter berdekatan)
tidak bertumpuk
OFFSET_LABEL = {"LeNet-5": (7, 4, "left"), "AlexNet": (-7, -13,
"right"), "VGG-16": (7, 6, "left"),
                "GoogLeNet": (7, 4, "left"), "ResNet-18": (7, -12,
"left"), "MobileNetV2": (7, 4, "left")}
fig, axs = plt.subplots(1, 3, figsize=(16, 4.6))

ax = axs[0]
bar = ax.barh(urut["Model"], urut["Test acc (%)"], color="#2a78d6",
height=0.6)
for b, v in zip(bar, urut["Test acc (%)"]):
    ax.text(v + 0.5, b.get_y() + b.get_height() / 2, f"{v:.2f}%",
```

```
va="center", fontsize=9)
ax.set_xlim(0, 100)
ax.set_title(f"Akurasi uji (epoch {CONFIG['EPOCHS']})")
ax.set_xlabel("%")
ax.grid(axis="y", visible=False)

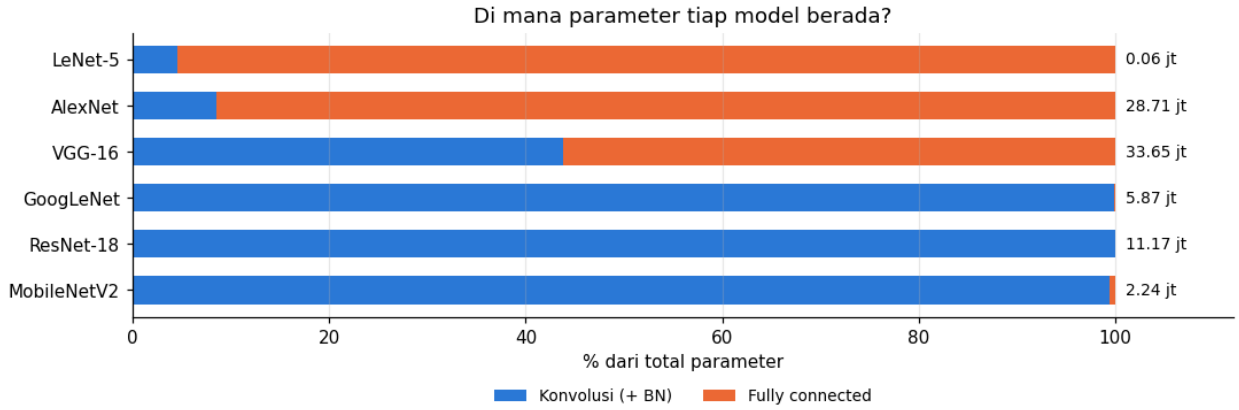
for ax, kolom, judul in [(axs[1], "Parameter (juta)", "Akurasi vs
jumlah parameter"),
                         (axs[2], "MAC (juta)", "Akurasi vs komputasi
per citra")]:
    for _, r in tabel.iterrows():
        ax.scatter(r[kolom], r["Test acc (%)"], s=70, color="#2a78d6",
marker=PENANDA[r["Model"]], zorder=3)
        dx, dy, ha = OFFSET_LABEL[r["Model"]]
        ax.annotate(r["Model"], (r[kolom], r["Test acc (%)"]),
xytext=(dx, dy), textcoords="offset points",
                    fontsize=9, ha=ha)
    ax.set_xscale("log")
    ax.set_title(judul)
    ax.set_xlabel(kolom + " — skala log")
    ax.set_ylabel("Akurasi uji (%)")
plt.tight_layout()
plt.savefig(os.path.join(CONFIG["DIR_HASIL"],
"akurasi_efisiensi.png"), bbox_inches="tight")
plt.show()
```



7.3 Distribusi parameter: konvolusi vs fully connected

```
fig, ax = plt.subplots(figsize=(10, 3.6))
conv_pct = 100 - tabel["% param di FC"]
ax.barh(tabel["Model"], conv_pct, color="#2a78d6", height=0.6,
label="Konvolusi (+ BN)")
ax.barh(tabel["Model"], tabel["% param di FC"], left=conv_pct,
color="#eb6834", height=0.6, label="Fully connected")
for i, (c, p) in enumerate(zip(conv_pct, tabel["Parameter (juta)"])):
    ax.text(101, i, f"{p:.2f} jt", va="center", fontsize=9)
ax.set_xlim(0, 112)
ax.invert_yaxis()
```

```
ax.set_xlabel("% dari total parameter")
ax.set_title("Di mana parameter tiap model berada?")
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.2), ncol=2,
fontsize=9, frameon=False)
ax.grid(axis="y", visible=False)
plt.tight_layout()
plt.savefig(os.path.join(CONFIG["DIR_HASIL"],
"distribusi_parameter.png"), bbox_inches="tight")
plt.show()
```

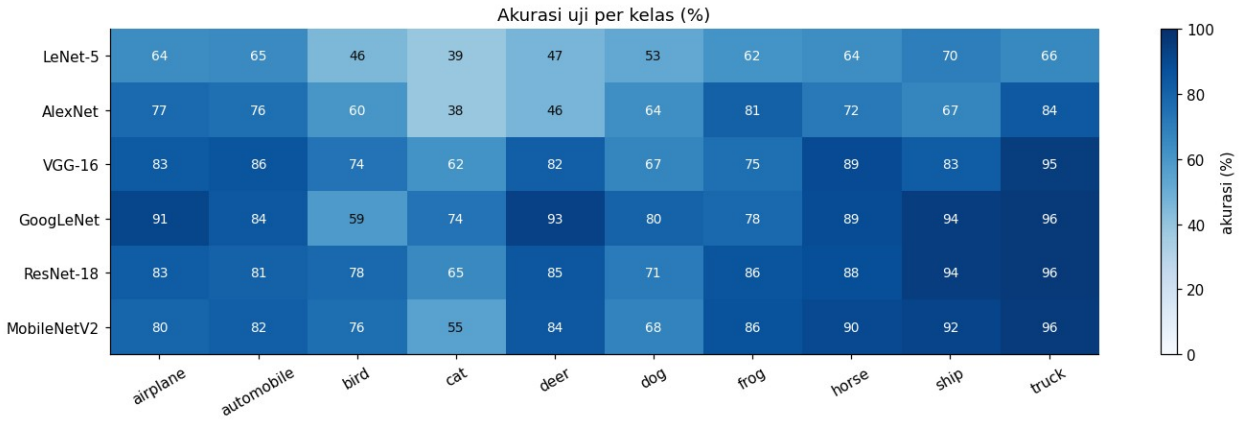


8. Analisis per Kelas

8.1 Akurasi per kelas untuk setiap model

```
y_np = y_test.numpy()
akurasi_kelas = np.array([[(np.array(HASIL[n]["pred_test"])[y_np == c]
== c).mean() * 100 for c in range(10)]
                          for n in MODEL_DIPAKAI])
fig, ax = plt.subplots(figsize=(13, 4.2))
im = ax.imshow(akurasi_kelas, cmap="Blues", vmin=0, vmax=100,
aspect="auto")
ax.set_xticks(range(10), KELAS, rotation=30)
ax.set_yticks(range(len(MODEL_DIPAKAI)), MODEL_DIPAKAI)
ax.grid(False)
for i in range(len(MODEL_DIPAKAI)):
    for j in range(10):
        v = akurasi_kelas[i, j]
        ax.text(j, i, f"{v:.0f}", ha="center", va="center",
fontsize=9, color="white" if v > 60 else "#0b0b0b")
fig.colorbar(im, ax=ax, label="akurasi (%)")
ax.set_title("Akurasi uji per kelas (%)")
plt.tight_layout()
plt.savefig(os.path.join(CONFIG["DIR_HASIL"],
"akurasi_per_kelas.png"), bbox_inches="tight")
plt.show()
```

```
rata_kelas = akurasi_kelas.mean(axis=0)
print("Kelas tersulit (rata-rata 6 model):", ", ".join(f"{KELAS[i]}
({rata_kelas[i]:.1f}%)" for i in rata_kelas.argsort()[:3]))
print("Kelas termudah (rata-rata 6 model):", ", ".join(f"{KELAS[i]}
({rata_kelas[i]:.1f}%)" for i in rata_kelas.argsort()[::-1][:3]))
```




```
Kelas tersulit (rata-rata 6 model): cat (55.7%), bird (65.5%), dog
(67.0%)
Kelas termudah (rata-rata 6 model): truck (88.7%), ship (83.4%), horse
(82.1%)
```

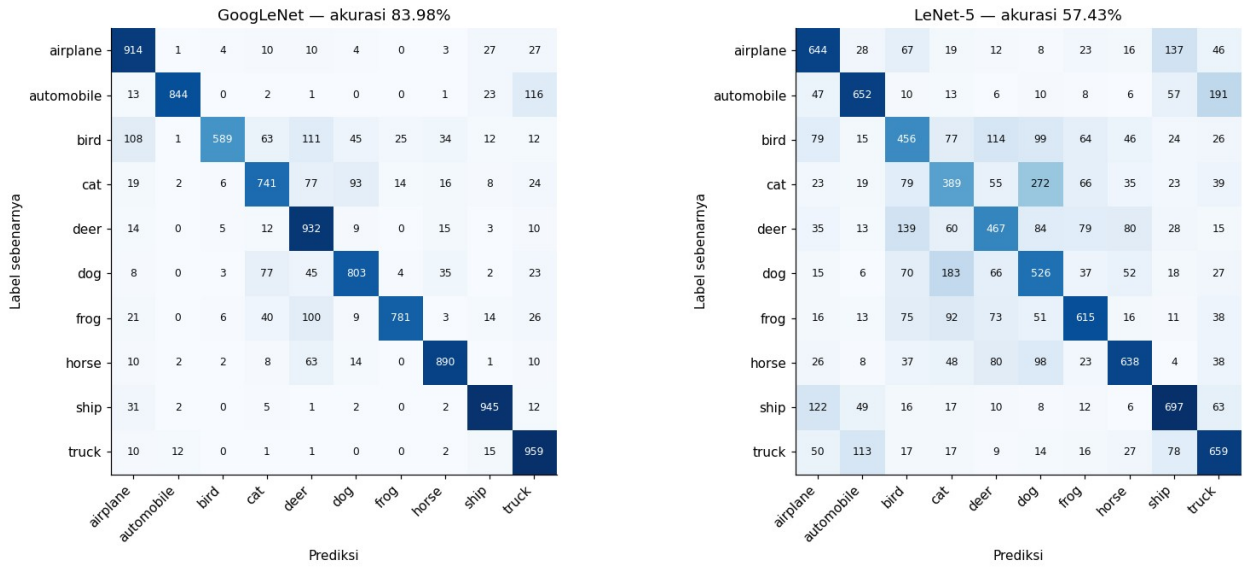
8.2 Confusion matrix: model terbaik vs model terlemah

```
def confusion(pred):
    cm = np.zeros((10, 10), dtype=int)
    for t, p in zip(y_np, pred):
        cm[t, p] += 1
    return cm

terbaik = tabel.loc[tabel["Test acc (%)"].idxmax(), "Model"]
terlemah = tabel.loc[tabel["Test acc (%)"].idxmin(), "Model"]
fig, axs = plt.subplots(1, 2, figsize=(15, 6.2))
for ax, nama in zip(axs, [terbaik, terlemah]):
    cm = confusion(HASIL[nama]["pred_test"])
    ax.imshow(cm, cmap="Blues")
    ax.grid(False)
    ax.set_xticks(range(10), KELAS, rotation=45, ha="right")
    ax.set_yticks(range(10), KELAS)
    for i in range(10):
        for j in range(10):
            ax.text(j, i, cm[i, j], ha="center", va="center",
fontsize=8, color="white" if cm[i, j] > cm.max() / 2 else "#0b0b0b")
    ax.set_xlabel("Prediksi")
    ax.set_ylabel("Label sebenarnya")
    ax.set_title(f"{nama} — akurasi {np.trace(cm) / cm.sum():.2%}")
plt.tight_layout()
```

```
plt.savefig(os.path.join(CONFIG["DIR_HASIL"], "confusion_matrix.png"),
bbox_inches="tight")
plt.show()

for nama in [terbaik, terlemah]:
    cm = confusion(HASIL[nama]["pred_test"])
    np.fill_diagonal(cm, 0)
    i, j = np.unravel_index(cm.argmax(), cm.shape)
    print(f"{nama}: kesalahan terbanyak = '{KELAS[i]}' diprediksi
sebagai '{KELAS[j]}' ({cm[i, j]} citra)")
```




```
GoogLeNet: kesalahan terbanyak = 'automobile' diprediksi sebagai
'truck' (116 citra)
LeNet-5: kesalahan terbanyak = 'cat' diprediksi sebagai 'dog' (272
citra)
```

9. Ringkasan Temuan (dihitung otomatis dari hasil)

Bagian ini hanya menyajikan **fakta numerik**. Interpretasinya ada di Bagian 10.

```
t = tabel.set_index("Model")
acc = t["Test acc (%)"]
E = CONFIG["EPOCHS"]
print(f"1. Peringkat akurasi uji (epoch {E}):")
for i, (n, v) in enumerate(acc.sort_values(ascending=False).items(),
1):
    print(f"   {i}. {n:<12} {v:6.2f}%  ({t.loc[n, 'Parameter
(juta)']:.2f} jt parameter, {t.loc[n, 'MAC (juta)']:.0f} jt MAC)")

base = acc["LeNet-5"]
print(f"\n2. Dibanding baseline LeNet-5 ({base:.2f}%):")
```

```
for n in MODEL_DIPAKAI[1:]:
    print(f"   {n:<12} {acc[n] - base:+6.2f} poin")

eff = acc / t["Parameter (juta)"]
best_eff = eff.drop("LeNet-5").idxmax()
print(f"\n3. Efisiensi parameter (di luar LeNet-5): {best_eff} memberi
akurasi terbanyak per juta parameter "
      f"({acc[best_eff]:.2f}% dengan {t.loc[best_eff, 'Parameter
(juta)']:.2f} jt).")
n_max, n_min = t["Parameter (juta)"].idxmax(), t.drop("LeNet-5")
["Parameter (juta)"].idxmin()
print(f"   {n_max} punya {t.loc[n_max, 'Parameter (juta)'] /
t.loc[n_min, 'Parameter (juta)']:.0f}x parameter {n_min}; "
      f"selisih akurasinya {acc[n_max] - acc[n_min]:+.2f} poin.")

print(f"\n4. Overfitting (gap train - test pada epoch {E}):")
for n, g in t["Gap train-test"].sort_values(ascending=False).items():
    label = "overfitting jelas" if g > 10 else ("overfitting ringan"
if g > 3 else "belum overfitting")
    print(f"   {n:<12} {g:+6.2f} poin -> {label}")

print(f"\n5. Apakah model masih belajar di akhir? (perubahan akurasi
uji epoch {E-2} -> {E})")
for n in MODEL_DIPAKAI:
    r = HASIL[n]["riwayat"]
    d = 100 * (r[-1]["test_acc"] - r[-3]["test_acc"])
    print(f"   {n:<12} {d:+6.2f} poin -> " + (f"masih naik: {E} epoch
belum cukup" if d > 1 else "sudah mendatar / berfluktuasi"))

gagal = [n for n in MODEL_DIPAKAI if acc[n] < 15]
print("\n6. Model yang gagal belajar (akurasi uji < 15%, setara
tebakan acak 10%):")
print("   " + (", ".join(gagal) + " -> bahas penyebabnya di Bagian 10
(mis. tanpa BatchNorm + Adam lr 0,001)" if gagal else "tidak ada"))

print("\n7. Biaya waktu training (rata-rata per epoch):")
for n, s in t["Waktu/epoch (s)"].sort_values().items():
    print(f"   {n:<12} {s:6.1f} s")
```

```
1. Peringkat akurasi uji (epoch 10):
   1. GoogLeNet     83.98%  (5.87 jt parameter, 1447 jt MAC)
   2. ResNet-18     82.64%  (11.17 jt parameter, 555 jt MAC)
   3. MobileNetV2   80.90%  (2.24 jt parameter, 88 jt MAC)
   4. VGG-16        79.64%  (33.65 jt parameter, 332 jt MAC)
   5. AlexNet       66.60%  (28.71 jt parameter, 205 jt MAC)
   6. LeNet-5       57.43%  (0.06 jt parameter, 1 jt MAC)

2. Dibanding baseline LeNet-5 (57.43%):
   AlexNet       +9.17 poin
   VGG-16       +22.21 poin
```

```
   GoogLeNet    +26.55 poin
   ResNet-18    +25.21 poin
   MobileNetV2  +23.47 poin

3. Efisiensi parameter (di luar LeNet-5): MobileNetV2 memberi akurasi
terbanyak per juta parameter (80.90% dengan 2.24 jt).
   VGG-16 punya 15x parameter MobileNetV2; selisih akurasinya -1.26
poin.

4. Overfitting (gap train - test pada epoch 10):
   ResNet-18    +14.33 poin -> overfitting jelas
   GoogLeNet    +10.19 poin -> overfitting jelas
   MobileNetV2   +9.16 poin -> overfitting ringan
   LeNet-5       +7.97 poin -> overfitting ringan
   VGG-16        +5.05 poin -> overfitting ringan
   AlexNet       +2.41 poin -> belum overfitting

5. Apakah model masih belajar di akhir? (perubahan akurasi uji epoch 8
-> 10)
   LeNet-5       +0.60 poin -> sudah mendatar / berfluktuasi
   AlexNet       -0.14 poin -> sudah mendatar / berfluktuasi
   VGG-16        +1.64 poin -> masih naik: 10 epoch belum cukup
   GoogLeNet     -0.42 poin -> sudah mendatar / berfluktuasi
   ResNet-18     -0.79 poin -> sudah mendatar / berfluktuasi
   MobileNetV2   -0.23 poin -> sudah mendatar / berfluktuasi

6. Model yang gagal belajar (akurasi uji < 15%, setara tebakan acak
10%):
   tidak ada

7. Biaya waktu training (rata-rata per epoch):
   LeNet-5        2.9 s
   AlexNet       15.4 s
   ResNet-18     24.5 s
   VGG-16        24.6 s
   MobileNetV2   30.5 s
   GoogLeNet     87.6 s
```

10. Analisis dan Pembahasan

Semua angka di bawah berasal dari keluaran Bagian 5–9 (satu seed = 42, 10 epoch, Adam lr 0,001, batch 64). Penjelasan mekanisme berasal dari materi Pertemuan 5. Bagian "dugaan" belum diuji dengan eksperimen ablasi.

10.1 Kapasitas model: LeNet-5 → AlexNet

LeNet-5 hanya punya ±62 ribu parameter dan 2 layer konvolusi dengan 6/16 filter. Akurasi ujinya **57,43%** dengan gap train-test **7,97 poin**. Akurasi latihnya sendiri hanya 65,40% dan test loss

mendatar di kisaran 1,20–1,22 sejak epoch 6. Jadi LeNet-5 **underfitting**: kapasitasnya tidak cukup untuk citra berwarna 10 kelas yang jauh lebih bervariasi daripada MNIST (98,6% di notebook 02).

AlexNet (±28,7 juta parameter, ReLU, Dropout) mencapai **66,60%**, naik **9,17 poin** dari LeNet-5. Kenaikan ini berasal dari kapasitas yang jauh lebih besar dan ReLU yang tidak saturasi seperti Tanh (slide 9). Akurasi latih AlexNet pun baru 69,01% (gap hanya 2,41 poin). Gap kecil di sini **bukan** tanda generalisasi bagus, melainkan model yang belum menyerap data latih dengan baik. Dugaan penyebabnya: Dropout 0,5 tanpa BatchNorm memperlambat belajar, dan max-pool 3×3 stride 2 memperkecil peta fitur terlalu cepat (32 → 15 → 7 → 3).

10.2 Kedalaman dan filter 3×3: AlexNet → VGG-16

VGG-16 menumpuk 13 conv 3×3. Dua 3×3 bertumpuk memberi receptive field 5×5 dengan bobot $18C^2$ dibanding $25C^2$ (slide 12), plus non-linearitas tambahan. Hasil: VGG-16 **79,64%** vs AlexNet **66,60%** (selisih **+13,04 poin**).

Perhatikan distribusi parameter (Bagian 7.3): AlexNet menyimpan **91,5%** parameternya di FC, sedangkan VGG-16 **56,2%**. Pada versi CIFAR, FC6 VGG hanya menerima 512 input (bukan 25.088 seperti versi ImageNet), sehingga porsi FC jauh lebih kecil dari 89% di slide 14. Meski VGG-16 punya parameter terbanyak (33,6 juta) dan ukuran terbesar (134,6 MB), parameter tambahan itu tidak menjadi penentu akurasi.

Kurva belajarnya khas: VGG-16 paling lambat di awal (test acc epoch 1 hanya 30,11%, epoch 3 baru 47,42%), lalu melonjak di epoch 4 (62,44%) dan epoch 7 (75,56%). Dugaan: jaringan 13 conv butuh beberapa epoch agar gradien dan bobot BatchNorm "menemukan" arah yang baik.

10.3 Lebar dan efisiensi: VGG-16 → GoogLeNet

GoogLeNet memakai ±5,87 juta parameter, ±5,7× lebih sedikit dari VGG-16, berkat bottleneck 1×1 dan Global Average Pooling yang menggantikan FC raksasa (slide 16–17). Hanya 0,17% parameternya berada di FC. Akurasinya **83,98%**, tertinggi dari keenam model.

Namun komputasinya (±1.447 juta MAC) justru yang **terbesar**: 4,4× VGG-16 dan 2,6× ResNet-18, karena modul Inception pertama bekerja di resolusi penuh 32×32. Ini terlihat dari waktu per epoch (**87,6 detik**, 3,6× ResNet-18) dan dari total 32,5 menit training, di mana GoogLeNet sendiri memakan sekitar 45%-nya. Pelajarannya: **parameter sedikit tidak sama dengan komputasi ringan**.

10.4 Residual connection: ResNet-18

ResNet-18 mencapai **82,64%** dengan ±11,17 juta parameter. Shortcut membuat gradien selalu punya jalur langsung (suku +1 pada slide 21), sehingga jaringan 18 layer dengan BatchNorm dapat dilatih tanpa gagal sejak epoch pertama. Konvergensinya termasuk paling cepat: di epoch 3 test acc ResNet-18 sudah **78,91%**, lebih tinggi dari GoogLeNet (75,73%), MobileNetV2 (74,94%), VGG-16 (47,42%), AlexNet (56,07%), dan LeNet-5 (53,23%). Di epoch 1 hanya GoogLeNet yang sedikit lebih tinggi (67,24% vs 64,77%).

Catatan: "stabil" di sini berarti tidak ada kegagalan training. Test accuracy ResNet-18 tetap berfluktuasi antar epoch (mis. turun ke 74,91% di epoch 5 sebelum kembali ke 82,98% di epoch 6). Penyebab yang wajar: learning rate tetap 0,001 tanpa scheduler. Waktu per epoch hanya 24,5 detik, sehingga ResNet-18 menawarkan rasio akurasi per waktu yang sangat baik.

10.5 Efisiensi untuk perangkat mobile: MobileNetV2

MobileNetV2 hanya memakai ±2,24 juta parameter (8,95 MB) dan ±88 juta MAC (**±16× lebih ringan** dari GoogLeNet, ±5× lebih sedikit parameter dari ResNet-18), tetapi mencapai **80,90%**. Depthwise separable convolution memangkas biaya konvolusi ±8–9× (slide 28–29). Selisih akurasinya dengan model terbaik hanya **3,08 poin** (≈96% dari akurasi GoogLeNet dengan ≈6% komputasinya). Trade-off ini sepadan untuk aplikasi di ponsel.

Temuan menarik: waktu per epoch MobileNetV2 (**30,5 detik**) justru lebih lama daripada ResNet-18 (24,5 detik) walau MAC-nya 6× lebih kecil. Dugaan: depthwise convolution punya intensitas aritmetika rendah sehingga kurang efisien di GPU (T4). Jadi **MAC bukan ukuran waktu nyata**; kecepatan sebenarnya bergantung pada perangkat keras.

10.6 Overfitting dan keterbatasan 10 epoch

Model dengan gap terbesar adalah **ResNet-18 (14,33 poin)**, disusul GoogLeNet (10,19) dan MobileNetV2 (9,16). Tanpa augmentasi data, model berkapasitas besar cenderung menghafal data latih. Pada ResNet-18, train acc 96,97% sedangkan test loss terendah ada di epoch 6 (0,538) lalu naik menjadi 0,700. Pada GoogLeNet test loss naik dari 0,499 (epoch 8) ke 0,620 (epoch 10), sementara akurasi uji hanya mendatar. Artinya model makin terlalu percaya diri pada prediksinya, dan akurasi uji maksimum keempat model terbaik dicapai di **epoch 8**, bukan epoch 10.

Gap saja tidak cukup untuk menilai. LeNet-5 dan AlexNet punya gap kecil karena akurasi latihnya sendiri rendah (65,4% dan 69,0%), yaitu **underfitting**, bukan generalisasi yang baik. Satu-satunya model yang akurasi ujinya masih naik di akhir adalah **VGG-16** (+1,64 poin dari epoch 8 ke 10, test loss 0,653 dan masih menurun). Model ini kemungkinan paling diuntungkan oleh tambahan epoch.

10.7 Kelas yang sulit

Kelas tersulit adalah **cat (rata-rata 55,7%)**, **bird (65,5%)**, dan **dog (67,0%)**; yang termudah **truck (88,7%)**, **ship (83,4%)**, dan **horse (82,1%)**. Kesalahan terbanyak model terbaik (GoogLeNet) adalah **automobile diprediksi sebagai truck (116 citra)**, dan model terlemah (LeNet-5) **cat diprediksi sebagai dog (272 dari 1.000 citra)**. Hal ini masuk akal: kucing dan anjing punya bentuk tubuh, bulu, dan latar yang serupa pada resolusi 32×32, sedangkan mobil dan truk sama-sama kendaraan beroda empat dengan siluet mirip. Kelas kendaraan umumnya lebih mudah karena bentuknya kaku dan latarnya khas (jalan, laut, langit).

Menariknya, GoogLeNet yang terbaik secara keseluruhan hanya 59% pada kelas *bird*, di bawah ResNet-18 (78%), MobileNetV2 (76%), dan VGG-16 (74%). Model dengan akurasi total tertinggi belum tentu terbaik di setiap kelas.

10.8 Efisiensi dan trade-off pada empat model utama

Tabel berikut merangkum empat model utama (LeNet-5, AlexNet, VGG-16, GoogLeNet). FLOPs ≈ 2 × MAC.

Model	Akurasi uji	Parameter	MAC	FLOPs	Ukuran	Detik/ epoch
GoogLeNet	**83,98%**	5,87 juta	1.447 juta	≈2,89 G	23,5 MB	87,6
VGG-16	79,64%	33,65 juta	332 juta	≈0,66 G	134,6 MB	24,6
AlexNet	66,60%	28,71 juta	205 juta	≈0,41 G	114,9 MB	15,4
LeNet-5	57,43%	0,06 juta	0,65 juta	≈1,3 juta	0,25 MB	2,9

Efisiensi: bergantung pada ukuran yang dipakai.

- **Parameter dan ukuran model: GoogLeNet.** Akurasi tertinggi dengan hanya 5,87 juta parameter (23,5 MB), 5,7× lebih kecil dari VGG-16 dan 4,9× lebih kecil dari AlexNet. Penyebabnya, GoogLeNet memakai bottleneck 1×1 dan Global Average Pooling (hanya 0,17% parameter di FC), sedangkan 91,5% parameter AlexNet dan 56,2% parameter VGG-16 menumpuk di FC.
- **FLOPs/MAC: LeNet-5 terendah secara mutlak**, tetapi akurasinya terlalu rendah untuk dianggap efisien; efisiensi yang berarti harus dilihat bersama akurasi. Di antara model yang akurasinya layak, **VGG-16** memberi hasil terbaik per komputasi: 79,64% dengan 332 juta MAC, yaitu ≈95% akurasi GoogLeNet dengan hanya ≈23% komputasinya.
- **AlexNet kurang efisien dari sisi parameter**: 28,7 juta parameter (hampir sama dengan VGG-16), tetapi akurasinya 13,04 poin lebih rendah dan jauh lebih besar dari GoogLeNet. Dari sisi komputasi AlexNet lebih ringan daripada VGG-16 (205 vs 332 juta MAC) dan paling cepat dilatih di antara ketiganya (15,4 detik/epoch), tetapi hemat komputasi itu tidak mengimbangi akurasinya yang jauh lebih rendah.

Trade-off utama yang teramati.

- **Akurasi vs komputasi (GoogLeNet).** Dibanding VGG-16, GoogLeNet menambah +4,34 poin, tetapi MAC-nya 4,4× lebih besar dan waktu training 3,6× lebih lama. Parameter sedikit tidak berarti komputasi ringan.
- **Akurasi vs ukuran model.** AlexNet dan VGG-16 memiliki 28–34 juta parameter, tetapi tambahan parameter di FC tidak menaikkan akurasi secara sepadan. GoogLeNet jauh lebih kecil namun lebih akurat.
- **Biaya komputasi vs kenaikan akurasi.** LeNet-5 → AlexNet menambah +9,17 poin dengan komputasi ≈315× lebih besar. AlexNet → VGG-16 menambah +13,04 poin dengan komputasi hanya ≈1,6× lebih besar. Kenaikan terbesar per biaya ada di langkah AlexNet → VGG-16.
- **Kapasitas vs generalisasi.** LeNet-5 dan AlexNet underfitting (train acc 65% dan 69%). GoogLeNet punya gap train-test 10,19 poin dan test loss yang naik di epoch 8–10, tanda mulai overfitting tanpa augmentasi.
- **Kecepatan konvergensi.** VGG-16 sangat lambat di awal (30% di epoch 1) dan satu-satunya model yang masih naik di epoch 10, sehingga hasilnya kemungkinan masih bisa membaik dengan epoch lebih banyak.

10.9 Keterbatasan eksperimen

1. **Dilatih dari nol, bukan pretrained.** Keputusan ini sengaja diambil agar keenam arsitektur dibandingkan dalam kondisi identik; pretrained akan mencampur pengaruh arsitektur

dengan pengaruh pretraining ImageNet. Adam lr 0,001 juga terlalu agresif untuk fine-tuning bobot pretrained.

2. **Satu seed (42) saja.** Selisih akurasi di bawah ±1–1,5 poin (mis. GoogLeNet vs ResNet-18, 1,34 poin) belum tentu signifikan; dibutuhkan beberapa seed. Operasi GPU dan mixed precision juga tidak 100% deterministik, sehingga run ulang bisa berbeda sedikit.
3. **10 epoch dengan learning rate tetap.** Hasil ini mengukur kecepatan belajar dalam anggaran pendek, bukan akurasi maksimum tiap arsitektur.
4. **Adaptasi 32×32.** Model yang dirancang untuk ImageNet telah diubah bagian stem-nya. VGG-16 dan GoogLeNet memakai BatchNorm, yang tidak ada di paper aslinya.
5. **Data uji dipantau setiap epoch** karena pembagian default tidak menyediakan data validasi. Yang dilaporkan sebagai metrik resmi adalah akurasi epoch ke-10 (peringkat tiga teratas tidak berubah bila memakai akurasi maksimum).
6. **Tanpa augmentasi data.** Ini memperbesar gap pada model besar (ResNet-18, GoogLeNet).

11. Kesimpulan

1. **Akurasi uji tertinggi: GoogLeNet (83,98%)**, terendah LeNet-5 (57,43%). Pada empat model utama urutannya GoogLeNet > VGG-16 (79,64%) > AlexNet (66,60%) > LeNet-5. Bila dua model pilihan disertakan, urutan lengkapnya: GoogLeNet > ResNet-18 (82,64%) > MobileNetV2 (80,90%) > VGG-16 > AlexNet > LeNet-5. Selisih GoogLeNet dengan VGG-16 (4,34 poin) lebih besar daripada fluktuasi akurasi antar epoch (±1–2 poin), tetapi tetap berasal dari satu seed.
2. **Akurasi tidak sebanding lurus dengan jumlah parameter.** AlexNet (28,7 juta) hanya 66,60%, sedangkan MobileNetV2 (2,24 juta) mencapai 80,90%. VGG-16 punya 15× parameter MobileNetV2 tetapi akurasinya 1,26 poin lebih rendah. Yang lebih menentukan adalah rancangan arsitektur: BatchNorm, GAP alih-alih FC besar, bottleneck 1×1, dan residual connection.
3. **Efisiensi (empat model utama).** GoogLeNet paling efisien dalam parameter dan ukuran (5,87 juta parameter, 23,5 MB). VGG-16 paling efisien dalam komputasi di antara model yang akurasinya layak (≈95% akurasi GoogLeNet dengan ≈23% MAC). LeNet-5 paling ringan secara mutlak tetapi akurasinya rendah, dan AlexNet kurang efisien dari sisi parameter dan akurasi.
4. **Trade-off utama.** (a) Akurasi vs komputasi: GoogLeNet menambah 4,34 poin dari VGG-16 dengan MAC 4,4× dan waktu training 3,6× lebih besar. (b) Parameter vs komputasi: jumlah parameter kecil tidak menjamin komputasi ringan. (c) Kapasitas vs generalisasi: model kecil underfitting, model besar mulai overfitting tanpa augmentasi. (d) MAC vs waktu nyata: MobileNetV2 punya MAC 6× lebih kecil dari ResNet-18 tetapi lebih lambat per epoch di GPU T4.
5. **Rekomendasi model.**
   - **GoogLeNet** untuk akurasi maksimum, terutama bila ukuran model dibatasi (23,5 MB) dan waktu training atau inferensi bukan kendala utama.
   - **VGG-16** bila komputasi atau waktu training terbatas dan akurasi ≈80% sudah cukup, dengan catatan ukuran modelnya besar (134,6 MB).
   - **LeNet-5** hanya layak sebagai baseline, dan **AlexNet** tidak direkomendasikan karena akurasinya jauh di bawah VGG-16 dan GoogLeNet dengan ukuran model yang sama-sama besar.

- **ResNet-18** (model pilihan) adalah yang paling seimbang secara keseluruhan: 82,64% (≈98% akurasi GoogLeNet) dengan hanya ≈38% MAC dan waktu per epoch 3,6× lebih cepat.
- **MobileNetV2** (model pilihan) paling tepat untuk perangkat terbatas: 2,24 juta parameter (8,95 MB) dan 88 juta MAC, dengan akurasi hanya 3,08 poin di bawah model terbaik.

6. **Pengembangan lanjutan:** augmentasi data (random crop dan flip), learning rate scheduler, *early stopping* sekitar epoch 8, lebih banyak epoch untuk VGG-16, dan pengulangan dengan beberapa seed untuk memastikan peringkat tiga teratas.
